0 8

SOL- — Tracker Scope-Coverage Gate

Field	Value
Revision	1
Last modified	2026-07-22T21:45:00Z
Rank	8
Closes	the corpus-3 scope-coverage gap — **no existing anchor checks this axis**; every anchor assumes the work IS in the tracker's scope
Seam	build (workspace-level gate)
POC	`poc/sol08_scope_coverage/` — GREEN 5/5, RED-first, real mini git repos in the fixtures
Anchors mechanized	none existing — this is the one genuinely NEW check the evidence demanded (corpus 3 §5 distillation 3)

. The measured problem

Corpus 3 kept an **exemplary** tracker for its root repo (0 recorded reopens; ~0.3% git-corrected; closures citing runtime evidence at high rate) — while **~80% of its commit volume ran untracked beside it**: all ~11,000 submodule commits map to **15** tracked items; its single largest repo (`helix_agent` , **2,790 commits — more than the root itself**) has no tracker at all and 3 tracked items; that untracked repo carries the full multi-attempt churn shape (34 fix commits on one feature over 7 weeks) invisible to any counter (corpus 3 §4.1). Tracker *discipline* and tracker *scope* are different failure axes — a perfect ledger over 20% of the work certifies nothing about the other 80%.

. The mechanism

The M2/R3.1 pattern (declare the required set independently of execution) applied to **repos**:

1. A checked-in **scope manifest** declares every repo in the workspace as `TRACKED` or `EXEMPT:<reason>` (third-party-vendored, fork-mirror — the corpus-3 classification's 48+99 repos are the legitimate exempt classes, enumerated never silent).
2. The gate **discovers repos from the filesystem** and refuses any repo on disk that the manifest does not name (`UNREGISTERED-REPO`) — the helix_agent hole: a 2,790-commit repo cannot sit invisibly beside the tracker, because its *existence* is the violation, before any commit-quality question.
3. For each `TRACKED` repo, the fraction of non-merge commits referencing a tracked-item id must meet a floor (`LOW-COVERAGE` names the repo + the numbers). Empty repos and empty workspaces are reported / `BLIND` , never silently green.

. POC results

RED → GREEN **5/5** against real `git init` fixtures: id-referencing repo passes; zero-reference tracked repo fails with numbers; on-disk-but-unmanifested repo fails `UNREGISTERED-REPO` ; exempt repo enumerated with its reason; empty workspace is blind (2), never 0.

. The failure this makes IMPOSSIBLE

On a wired workspace, work can no longer accumulate in a repo the tracker cannot see: a new repo either enters the manifest (tracked or exempt-with-reason) or fails every build; a tracked repo's linkage ratio is a printed number every build, not an archaeology finding.

. What it still does NOT catch (honest boundary)

1. **Commit-message id references are a proxy** (§11.4.201(8) honesty): citing an id proves linkage *syntax*, not that the commit genuinely belongs to that item. Gaming it requires actively writing false ids — which converts silent scope-drift into visible fraud, a different (and rarer) failure class. The metric reaches its target at the correct end-state (all work genuinely item-linked ⇒ 100%), so it is a valid necessary floor.
2. **The threshold and window are consumer data** — corpus 3's own history (89% `test@example.com` author identity) shows per-author attribution is unrecoverable there; this gate deliberately keys on message content, not authorship. Identity hygiene remains a separate precondition (corpus 3 §5 distillation 4).
3. **Depth-2 discovery** in the POC; a deep submodule forest needs `git submodule status --recursive` as the discovery instrument at integration (stated; the corpus-3 analysis used exactly that and it is proven workable at 224 repos).